

시스템 도식화

[사용자]

|

| 자연어 요청 / 코드 수정 요청 / 문서 기반 질의



[IDE 확장/클라이언트]

|— 현재 파일, 선택 영역, 커서 위치 수집

|— 프로젝트 루트/열린 파일/대화 세션 정보 전달

달

|— 결과 표시(설명, diff, patch, 실행 결과)



[Agent Orchestrator]

|— 대화 상태 관리

|— 작업 계획 수립(ReAct/Tool-using)

|— 어떤 데이터 접근 방식이 필요한지 결정

| |— 코드 내부 접근 → line-span / symbol selector

| |— 외부 정적 데이터 → source pointer + extraction tool

| |— 큰 중간 결과 → runtime pointer + mirrored tool

|— 최종 응답/수정안 생성



[Local LLM Backend]

- └─ 기본 추론 모델(Qwen 계열 등)
 - └─ 선택적 Self-Extend 추론 레이어
 - └─ 긴 문맥에서 neighbor window + grouped attention 적용



[Pointer & Tool Layer]

- └─ Code Pointer Manager
 - └─ file-level pointer
 - └─ line-span selector
 - └─ symbol selector
- └─ Code Indexer / Parser
 - └─ file outline 생성
 - └─ symbol index 생성
 - └─ 함수/클래스 시작줄·끝줄 JSON 저장
- └─ Source Pointer Resolver
 - └─ PDF / JSON / CSV / 로그 / HTML / 정책문서 등 resolve
- └─ Runtime Object Store
 - └─ 큰 tool output 저장
 - └─ memory path 발급
 - └─ key 단위 계층 경로 관리

- |— Mirrored Tool Wrapper
 - | |— 입력이 raw인지 pointer인지 판별
 - | |— pointer resolve
 - | |— 원본 tool 호출
 - | |— 큰 출력이면 저장 후 pointer 반환
- |— Domain Tools / MCP Tools
 - |— read_span / read_symbol / write_patch
 - |— grep_log / extract_doc_section / extract_json_field
 - |— search_internal_docs / sql_assist / build / test
 - |— retrieve_final_answer_from_memory



[실제 데이터 계층]

- |— 로컬 프로젝트 소스코드
- |— symbol index JSON / file outline JSON
- |— 외부 문서(PDF, md, HTML, CSV, JSON, 로그)
- |— 사내 MCP 도구/지식베이스
 - |— runtime memory store

-흐름-

1. 사용자가 IDE에서 요청

"이 프로젝트에서 로그인 관련 버그를 찾아 수정해줘"

2. IDE가 현재 파일/선택영역/프로젝트 루트를 agent에 전달

3. Agent가 작업 유형 판단

└─ 코드 구조 파악 필요

└─ 관련 파일 탐색 필요

└─ 특정 함수 단위 수정 필요

4. Code Indexer가 file outline / symbol index 조회

└─ login(), validateToken(), AuthService 클래스 등 후보 반환

5. Agent가 read_symbol / read_span 호출

└─ 필요한 코드 일부만 읽음

6. 필요시 외부 문서나 로그도 조회

└─ source pointer 확보

└─ grep_log / extract_doc_section 등 최소 tool 호출

7. 큰 tool output이 나오면

- └─ Runtime Object Store에 저장
- └─ runtime pointer만 LLM에 전달

8. LLM이 수정 계획 생성

9. write_patch / replace_symbol 등으로 실제 코드 수정

10. 최종 출력

- └─ 무엇을 왜 고쳤는지 설명
- └─ diff/patch 제시
- └─ 필요시 테스트 결과 반환

-구조-

단계1: 코드 어시스턴트

line-span selector, symbol index, symbol selector, 코드 읽기/수정 tool을 먼저 완성한다. 이걸 외부 데이터 tool 없이도 실험 가능하다.

단계2: **source pointer**

긴 로그 grep, 긴 문서 section 추출, JSON field 추출 같은 최소 extraction tool을 붙여 외부 파일 실험을 한다.

단계3: **runtime pointer**

runtime object store, pointer resolver,

mirrored tool wrapper를 붙여 큰 중간 결과를 다음 tool로 넘기는 논문형 실험을 수행한다.

-컴포넌트-

입력

- 사용자 자연어 요청
- IDE 문맥(활성 파일, 선택 영역, 커서 위치, 프로젝트 루트)
- 로컬 프로젝트 코드베이스
- 외부 정적 데이터(PDF, 로그, JSON, CSV, HTML, 문서)
- MCP 기반 사내 도구
- 이전 tool output / runtime memory pointer

핵심 컴포넌트

- IDE Extension
- Agent Orchestrator
- Local LLM Backend
- Self-Extend Inference Layer(선택)
- Code Pointer Manager
- Code Indexer / Symbol Index / File Outline
- Source Pointer Resolver
- Runtime Object Store
- Mirrored Tool Wrapper
- Domain-specific Tools / MCP Connectors

출력

- 자연어 설명 및 최종 답변
- 코드 patch / diff / 파일 수정 결과
- symbol index / outline / memory path 같은

내부 산출물

- 검색 결과, 문서 추출 결과, SQL 초안, 테스트/
빌드 결과